

Project Blueprint: FPGA-Based CNN Accelerator with RISC-V SoC Integration

Team Technical Alignment Document

2-Month Execution Roadmap

Abstract

This document outlines the core architecture, theoretical foundations, and week-by-week responsibilities for our 2-month engineering project. Our objective is to design a hardware accelerator for Convolutional Neural Network (CNN) inference in SystemVerilog, integrated as a memory-mapped peripheral with an open-source RISC-V soft-core CPU on an FPGA.

1 The Core Objective: What Are We Accelerating?

Standard general-purpose CPUs execute operations sequentially, creating a massive bottleneck when computing dense matrix multiplications. A Convolutional Neural Network (CNN) spends over 90% of its execution time performing 2D convolutions (sliding a 3×3 filter of weights across a grid of pixels, multiplying them, and accumulating the results).

- **Software Approach (CPU):** Fetches one pixel/weight at a time, multiplies, adds, and loops millions of times.
- **Hardware Acceleration Approach (Our FPGA Design):** Uses **spatial parallelism**. We build hardwired multiplier arrays that compute an entire 3×3 matrix multiplication simultaneously on a single clock edge, drastically reducing execution time and enabling edge-AI inference.

2 System Architecture: How the Whole System Works

Rather than an isolated module tested only in a simulation waveform, our project forms a complete System-on-Chip (SoC) workflow:

1. **CPU Initialization (Software):** A RISC-V soft-core processor (e.g., PicoRV32) boots up and loads raw image pixels and weights into system memory.
2. **Control Hand-off:** The CPU uses a memory-mapped bus interface (such as AXI-Lite) to write configuration commands (e.g., start, base addresses) to our accelerator's control registers.
3. **Hardware Acceleration (Dataflow + Compute):** Our accelerator reads the pixel stream, feeds it through FPGA Block RAM (BRAM) line buffers to form a spatial 3×3 window, processes it via parallel Multiply-Accumulate (MAC) units, applies ReLU, and outputs the resulting feature map.
4. **Completion:** The accelerator asserts a done interrupt back to the RISC-V CPU, which then reads the processed feature map.

3 Essential Theory: What Everyone Must Study First

Before touching SystemVerilog code, every team member must understand these four pillars:

1. **2D Convolution Math:** Manually trace a 3×3 filter sliding across a matrix on paper to understand how numbers are multiplied and accumulated into a single feature map value.
2. **Fixed-Point Arithmetic:** Understand Q-format fixed-point representations (e.g., INT8/INT4) to avoid heavy floating-point logic overhead on the FPGA.
3. **Streaming Handshakes:** Master the concept of ‘TVALID’ and ‘TREADY’ signals to govern data flow and prevent buffer overruns.
4. **Memory-Mapped I/O:** Understand how a CPU reads/writes peripheral registers to control hardware blocks.

4 Team Role Distribution (Vertical Pillar Ownership)

To ensure continuous parallel progress over 8 weeks, responsibilities are divided across three dedicated domains:

Member	Domain	Core Responsibilities
Member 1	Datapath & Arithmetic (35%)	MAC units, multipliers, adders, ReLU activation, pooling.
Member 2	Dataflow & Memory (35%)	FPGA BRAM line buffers, 3×3 sliding window, padding logic.
Member 3	SoC Integration & Verification (30%)	Control FSM, RISC-V bus interface, Python co-simulation, Quartus timing.

5 Phase-Wise Execution Roadmap

Phase 1: Foundations & Core Components (Month 1: Weeks 1–4)

- **Week 1:** Architecture specs, primitive INT8 MAC design (Member 1), BRAM line buffer template (Member 2), Git repo setup & Python test vector generator (Member 3).
- **Week 2:** Hardware ReLU and max-pooling logic (Member 1), 3×3 sliding window stream generator (Member 2), SystemVerilog testbench infrastructure with \$readmemh (Member 3).
- **Week 3:** Arithmetic processing pipeline assembly (Member 1), vision streaming core integration (Member 2), Top-Level FSM control skeleton (Member 3).
- **Week 4:** Sub-system verification, local self-checking testbenches, and initial scratchpad module integration.

Phase 2: System Integration FPGA Sign-Off (Month 2: Weeks 5–8)

- **Week 5:** Pipeline register optimization (Member 1), pixel padding logic (Member 2), master top-level module assembly connecting the RISC-V CPU and accelerator (Member 3).
- **Week 6:** Arithmetic bug fixing (Member 1), memory handshake validation (Member 2), automated Python-to-SV co-simulation pipeline execution (Member 3).

- **Week 7:** Quartus resource utilization analysis (Member 1), BRAM inference check (Member 2), SDC timing constraints and f_{max} optimization (Member 3).
- **Week 8:** Performance documentation (Member 1), power analysis reports (Member 2), final README compilation and presentation prep (Member 3).